

OPEN SOURCE NODE.JS ARCHITECTURE

batch-rate-queue

The Zero-Redis Throttling & SQL Write-Buffering Layer. Built-in adaptive throttling, multi-tenant fair scheduling, cost-weighted token budgeting, and PostgreSQL distributed coordination.

Package: batch-rate-queue Version: v1.3.0 Runtime: Node.js >= 16 License: MIT Tests: 141 / 141 Passing

1 Real Production Gaps in Existing Queue Systems

Traditional background job queues (BullMQ, pg-boss, Celery) were built to distribute compute across worker pools. When modern applications interact with rate-limited APIs and relational databases, developers hit severe architectural walls:

1. Static Rates & Downstream Struggles

When downstream APIs encounter latency spikes, 429s, or 503s, queues like pg-boss and BullMQ continue hammering at static configured rates, piling up callbacks and crashing services.

2. Single-Dimension Limiting (LLM Token Gap)

Modern AI APIs (OpenAI, Anthropic) limit by **tokens per minute (TPM)**, not requests. General queues treat 1 job = 1 unit, unable to budget variable prompt/completion weights.

3. Multi-Tenant Starvation & Paid Paywalls

One tenant enqueueing 50k jobs blocks everyone else. BullMQ removed group rate limiting in v3 and made it paid-only. No free, robust multi-tenant scheduler existed.

4. The Redis Tax & Database Contention

Forcing Redis infrastructure (\$20-\$100+/mo) for rate limiting, while naive workers fire 10,000 individual SQL UPDATES, exhausting DB connection pools and CPU locks.

2 The "Zero-Redis" Architectural Paradigm

The Dedicated Throttling & Write-Buffering Layer

Instead of replacing your existing orchestrator, batch-rate-queue acts as the specialized **throttling and write-buffering layer** that sits in front of whatever executes your jobs. It runs purely in-process with optional PostgreSQL distributed coordination across pods—eliminating Redis entirely.

0 REDIS CLUSTERS NEEDED	98.5% DB ROUNDTRIP REDUCTION	100% TENANT STARVATION FREE (DRR)	0 B DATA LOST ON SIGTERM
----------------------------	---------------------------------	--------------------------------------	-----------------------------

3 Complete Architecture Blueprint

Incoming tasks flow through a multi-stage pipeline guaranteeing fair scheduling, token budgeting, adaptive backoff, and transactional batch writing:



4 Six Pillar Capabilities

1. Reactive / Adaptive Throttling

Sliding-window monitor reads live 429s, Retry-After headers, and rising error rates. Multiplies rate by backoffFactor and recovers smoothly.

2. Cost-Weighted LLM Token Budgets

Enforces token-budget caps (e.g. 40k TPM). Each task consumes its estimated/actual token weight rather than a flat 1 request.

3. Multi-Tenant Fair-Share (DRR)

Deficit Round Robin scheduling isolates tenant queues. A tenant with 50,000 items will never starve a tenant with 5 items.

4. Zero-Redis Distributed Limiting

Optional PgTokenBucket uses PostgreSQL row locks and time-delta refills to coordinate global rate limits across Kubernetes pods.

5. PostgreSQL Distributed Circuit Breaker

Coordinates breaker state across all pods. When an API goes down, all replicas trip immediately without hammering the destination.

6. Pluggable Classifiers & Live Controls

Built-in error classifiers for HTTP 429/503 and OpenAI/Anthropic errors. Live runtime reconfiguration via setRateLimit().

5 Comprehensive Feature Comparison

Feature Capability	BullMQ	pg-boss	batch-rate-queue
Redis Dependency	Mandatory Redis Cluster	Zero (Postgres only)	Zero (Standalone or Postgres)
Write-Buffering (Batch SQL)	None (Row-by-row updates)	None (Row-by-row updates)	Native In-Memory Dual-Trigger Buffer
Reactive / Adaptive Throttling	Static Fixed Limiter	Static Rate Limiter	Live 429 & Retry-After Auto Backoff
Cost-Weighted Token Budgets	No (1 job = 1 token)	No (1 job = 1 token)	Yes (Per-job variable token weight)
Per-Key Multi-Tenant Isolation	Paid-only Pro tier	No native per-key buckets	Built-in KeyedThrottler (Free/MIT)
Fair-Share Tenant Scheduling	FIFO single queue	FIFO single queue	Deficit Round Robin (DRR) Fair Share
Distributed Circuit Breaker	Requires external tool	None	Postgres Shared Circuit Breaker
Runtime Limit Reconfiguration	Requires worker restart	Requires worker restart	Live setRateLimit() without restart

6 Code Recipe: Multi-Tenant Fair Scheduling

Isolate rate limits per tenant and prevent tenant starvation using Deficit Round Robin fair scheduling:

```
multi-tenant-worker.ts TypeScript

import { createBatchRateQueue } from 'batch-rate-queue';

const queue = createBatchRateQueue({
  name: 'tenant-sync-service',
  ratelimit: { requests: 10, perMs: 1000 }, // Default SLA: 10 req/sec
  batchFlush: { size: 100, intervalMs: 2000 },

  // 1. Isolate rate buckets by tenant ID
  ratelimitKey: (item) => item.tenantId,

  // 2. SLA Tier Overrides
  perKeyRateLimit: {
    'enterprise-corp': { requests: 100, perMs: 1000 },
    'free-tier-user': { requests: 2, perMs: 1000 },
  },

  // 3. Deficit Round Robin: 10k items from tenant A will NOT block tenant B
  fairShare: true,

  worker: async (item) => {
    const data = await syncExternalApi(item);
    return { id: item.id, updates: data };
  },
  onBatchFlush: async (batch) => db.bulkUpdate(batch),
});

// Live update without restarting workers
queue.setKeyRateLimit('free-tier-user', { requests: 5, perMs: 1000 });
```

7 Code Recipe: LLM Token-Budget Limiting (TPM)

Enforce token-based budgets (e.g. 40,000 tokens/min) for OpenAI or Anthropic API calls with pre-acquisition and actual-cost reconciliation:

```
llm-embeddings-worker.ts TypeScript

import { createBatchRateQueue, llmApiClassifier } from 'batch-rate-queue';
import OpenAI from 'openai';

const openai = new OpenAI();

const queue = createBatchRateQueue({
  name: 'openai-embeddings',
  // Budget: 40,000 tokens per minute
  ratelimit: { requests: 40000, perMs: 60000, costBased: true },
  batchFlush: { size: 50, intervalMs: 3000 },

  // Estimate token cost before acquire
  costExtractor: (item) => Math.ceil(item.text.length / 4),
  errorClassifier: llmApiClassifier,

  worker: async (item) => {
    const res = await openai.embeddings.create({
      model: 'text-embedding-3-small',
      input: item.text,
    });
    return {
      id: item.id,
      updates: { embedding: res.data[0].embedding },
      meta: { actualCost: res.usage.total_tokens }, // Exact actual cost
    };
  },
  onBatchFlush: async (batch) => db.saveVectors(batch),
});
```

8 Code Recipe: Zero-Redis Distributed PostgreSQL Coordination

Coordinate global rate limits and shared circuit breaking across 10+ Kubernetes replicas using PostgreSQL:

```
distributed-worker.ts

import { createBatchRateQueue, httpRateLimitClassifier } from 'batch-rate-queue';
import { Pool } from 'pg';

const pool = new Pool({ connectionString: process.env.DATABASE_URL });

const queue = createBatchRateQueue({
  name: 'stripe-billing-sync',
  ratelimit: { requests: 25, perMs: 1000 }, // Global 25 req/sec across ALL pods
  batchFlush: { size: 50, intervalMs: 2000 },

  // 1. PostgreSQL Shared Token Bucket (No Redis!)
  distributed: {
    pool,
    bucketKey: 'global:stripe-metering',
  },

  // 2. PostgreSQL Shared Circuit Breaker
  circuitBreaker: {
    pool,
    breakerKey: 'stripe-api',
    failureThreshold: 5, // Trip after 5 consecutive failures
    cooldownMs: 30000, // 30s cooldown before trial request
  },

  // 3. Reactive Adaptive Throttling
  adaptiveThrottle: {
    enabled: true,
    errorRateThreshold: 0.2, // Back off if >20% failures
    backoffFactor: 0.5,
    honorRetryAfter: true,
  },
  errorClassifier: httpRateLimitClassifier,

  worker: async (item) => stripe.charges.create(item.payload),
  onBatchFlush: async (batch) => db.saveCharges(batch),
});

queue.on('circuitBreakerTripped', (e) => {
  console.warn(`[ALERT] Circuit breaker ${e.breakerKey} TRIPPED!`);
});
```

9 Real-Time Monitoring & Dashboard Stats

Feed real-time metrics into Prometheus, Grafana, or your /metrics health endpoint via getBudgetStats():

```
metrics-endpoint.ts

app.get('/metrics/queue', (req, res) => {
  const stats = queue.getBudgetStats();
  res.json({
    queueState: stats.queue.state,
    pendingItems: stats.queue.pending,
    tokenUtilization: `${stats.ratelimit.utilizationPct}%`,
    effectiveRate: `${stats.ratelimit.effective.requests} req / ${stats.ratelimit.effective.perMs}ms`,
    backlogSecondsRemaining: stats.backlogEta.estimatedSecondsRemaining,
    estimatedCompletionTime: stats.backlogEta.estimatedCompletionTime,
    perKeyStats: stats.keys,
    circuitBreaker: stats.circuitBreaker,
  });
});
```

10 Native Database Adapters

Prisma Adapter

```
import { createPrismaFlushHandler }
  from 'batch-rate-queue';

const onFlush =
  createPrismaFlushHandler(
    prisma, 'location'
  );
```

PostgreSQL (pg)

```
import { createPgFlushHandler }
  from 'batch-rate-queue';

const onFlush =
  createPgFlushHandler(
    pool, 'locations'
  );
```

SQLite

```
import { createSqliteFlushHandler }
  from 'batch-rate-queue';

const onFlush =
  createSqliteFlushHandler(
    db, 'locations'
  );
```

11 Full Options Reference Table

Property	Type	Default	Description
name	string	required	Queue identifier for logs, metrics, and default bucket keys.
rateLimit	RateLimitConfig	required	Token bucket rate limit (requests, perMs, costBased).
batchFlush	BatchFlushConfig	required	Transactional write buffer trigger (size, intervalMs).
worker	Function	required	Worker execution per item returning { id, updates, meta }.
onBatchFlush	Function	required	Atomic transaction write callback (or pre-built DB adapter).
rateLimitKey	(item) => string	undefined	Isolates token buckets per tenant, API key, or destination.
fairShare	boolean	false	Enables Deficit Round Robin scheduling across tenant queues.
perKeyRateLimit	Record<string, Rate>	undefined	Custom SLA rate limits per tenant/key.
costExtractor	(item) => number	() => 1	Pre-acquire token budget estimator (for LLM tokens/TPM).
adaptiveThrottle	AdaptiveConfig	undefined	Automatic backoff on 429 / Retry-After and smooth recovery.
distributed	DistributedConfig	undefined	PostgreSQL-backed shared token bucket (zero-Redis multi-pod).
circuitBreaker	CircuitBreakerConfig	undefined	PostgreSQL-backed cross-process failure circuit breaker.

12 Official Ecosystem Resources

- **npm:** <https://www.npmjs.com/package/batch-rate-queue>
- **GitHub:** <https://github.com/piyushkumar-prog/batch-rate-queue>
- **License:** MIT (Open Source, Free for Commercial Use)
- **Author:** Piyush Kumar (@piyushkumar-prog)